

CSC 4553

Fall 2026 - Day 4

Admin

- Lab 2 due Friday, 18:00
- Quiz 2 due Monday, 18:00

Kernel basics

Questions to consider

- What are the different kernel paradigms?
- Why would you choose one over the other?

Paradigms

- The OS offers a number of services. What should go in the kernel?
 - IPC
 - VFS
 - File system
 - Scheduler
 - Virtual Memory
 - Device drivers

Monolithic kernels

- Oldest, very common design (Linux, Windows 9x, BSDs)
- Single piece of code in memory
- Limited **information hiding**
 - One part of the kernel can directly access data and functions of other parts

Monolithic kernels (cont'd)

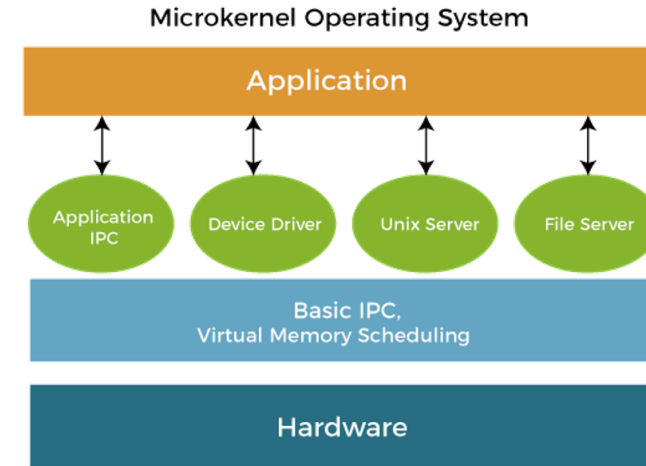
- Q: What happens when you need something new supported (new hardware device, new system call, new filesystem)?
- **Modules** (loadable kernel modules - LKMs) allow for flexibility, customization, support
 - Pros?
 - Memory savings
 - Flexibility
 - Minimal downtime
 - Cons?
 - (minor) fragmentation: the base kernel can be loaded contiguously in memory
 - Security (<https://github.com/m0nad/Diamorphine> **DO NOT USE THIS, IT'S SIMPLY TO SHOW YOU**)

Layered kernels

- Dijkstra created the THE OS in the 60s, introducing the concept Each inner layer is more privileged; required a trap to move down layers
- Hardware-enforcement possible
- Intel **announced** in May 2024 that the new architectures will only have rings 0 and 3

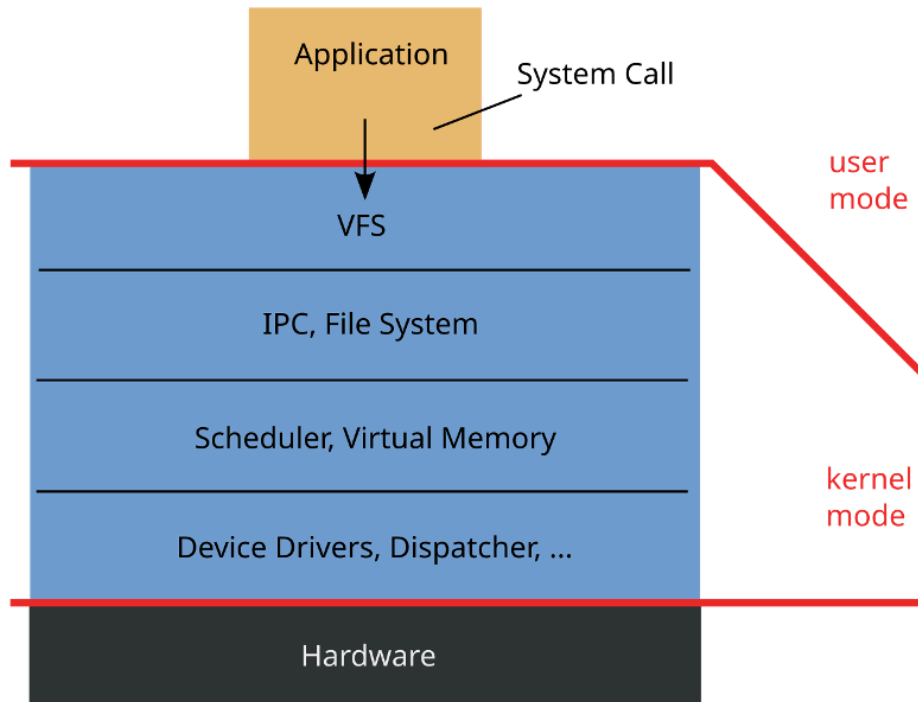
Microkernels

- Popular research area long ago, didn't win (although people remain interested in the principles)
- All non-essential components removed from the kernel, for modularization, extensibility, isolation, security, and ease of management
- A collection of OS services running in user space
- Downsides?
 - **Heavy communication costs** through message passing (marshaled through the kernel)

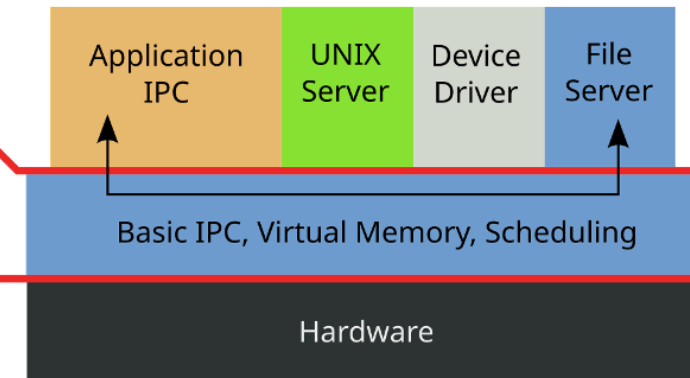


Monolithic vs Microkernel

Monolithic Kernel based Operating System



Microkernel based Operating System



Hybrid kernels

- Combine a monolithic core with microkernel ideas
- Small kernel handles critical services (scheduling, IPC, low-level memory)
- Other services (file systems, drivers) can run in kernel *or* user space
- Examples: macOS (XNU), Windows NT
 - XNU: Mach microkernel + BSD monolithic layer
 - Windows NT: microkernel-inspired, but most services run in kernel mode for performance
- Pragmatic compromise: **microkernel isolation where it matters, monolithic speed where it doesn't**

What isn't clear?

Comments? Thoughts?

Dual-mode / Syscalls

Questions to consider

- How do we ensure that a user process doesn't harm others?
- How do system calls work? How do they relate to wrapper libraries like `glibc`?

Interrupts and traps

- How does the OS regain control of the CPU when a process is running?
- Two mechanisms that transfer control to the kernel:
 - **Hardware interrupts**: asynchronous, generated by devices
 - Timer, disk I/O completion, keyboard, network
 - CPU checks for pending interrupts between instructions
 - **Software traps**: synchronous, generated by the running program
 - System calls (intentional)
 - Exceptions: divide-by-zero, page fault, illegal instruction (unintentional)
- Both use the same basic mechanism: save state, jump to a kernel handler via an **interrupt vector table**

Interrupts and traps (cont'd)



Why this matters

- Without hardware interrupts, a misbehaving process could monopolize the CPU forever
- The **timer interrupt** is the foundation of preemptive scheduling (we'll talk about scheduling later)
 - Hardware timer fires periodically (e.g., every 1-10 ms)
 - Forces a trap into kernel mode regardless of what the process is doing
 - Kernel's scheduler decides: resume this process or switch to another?
- This is the answer to: "if the process has the CPU, who stops it?"
 - **The hardware stops it**

Dual-mode operation

- Dual-mode operation allows OS to protect itself and components
 - **User mode** and **kernel** mode
- Mode bit provided/enforced by hardware
 - Provides ability to distinguish when system is running user code or kernel code.
 - When a user is running → mode bit is “user”
 - When kernel code is executing → mode bit is “kernel”
- **System call** changes mode to kernel, return from call resets it to user
- Some instructions are only executable in kernel mode

Hardware support for dual-mode

- The mode bit by itself doesn't protect anything — it's just a flag
- The hardware has to enforce four things:
 1. A **mode bit** that user code cannot set on its own
 2. **Privileged instructions** that only work in kernel mode
 3. **Memory protection** so a process can't touch kernel memory
 4. A **timer** so the kernel always gets the CPU back
- Remove any one of them and the protection falls apart

The mode bit and traps

- The mode bit lives inside the CPU, not in memory the process can write to
 - So a process can't just flip it to "kernel"
- The **only** way into kernel mode is a trap
- A trap does two things **at the same time**:
 - Switches the mode bit to kernel
 - Jumps to an address the **kernel** picked in advance (the interrupt vector table)
- That second part is the whole trick
 - If you could pick the address, you'd get kernel mode *and* run your own code

Privileged instructions

- Some instructions only work in kernel mode; try them in user mode and you get a trap instead
- Roughly, anything that would let you escape the sandbox:
 - Switching back to user mode (returning from a trap)
 - Changing where the interrupt vector table points
 - Changing the memory mapping for a process
 - Talking to devices directly
 - Turning off interrupts
- Q: why does “turn off interrupts” have to be privileged?

Memory protection

- The mode bit controls which *instructions* you can run
- It says nothing about which *memory* you can touch — that's a separate mechanism
- Every memory access goes through hardware that checks the page table
 - Pages are marked “kernel only” or “user OK”
 - Touching a kernel page from user mode is a trap (this is a **segfault**)
- And changing that mapping is itself a privileged instruction
 - Otherwise you'd just remap the kernel into your own address space

The timer

- A piece of hardware that fires an interrupt every few milliseconds, no matter what the CPU is doing
- That interrupt is a trap, so it puts the kernel back in control
- Setting the timer (or turning it off) is privileged — only the kernel can do it
- Without it, the kernel only runs when a process *chooses* to ask for something
 - Protection that depends on the good behavior of the program you're protecting against isn't protection
- We'll come back to what the kernel *does* with that control when we cover scheduling

System calls

- The OS offers a number of services. How do we (applications) interface with them?
 - We don't want to deal with the details, just the abstraction
 - The OS has ultimate control over these operations
- System calls are the “language” of communication with the OS
- Standards
 - Win32 (MS)
 - POSIX (nearly all Unix-based systems)

System calls (cont'd)

- We call into the library wrapper (e.g., `glibc`) which places arguments into registers
- Each system call has a special number that identifies it to the kernel
- Executes a TRAP instruction (switch to kernel mode)
- A logical separation of memory space
- Kernel's system call handler is invoked, once done (but may block) may be returned to the process

System calls (cont'd)

- Table defined in the kernel:
https://github.com/torvalds/linux/blob/master/arch/x86/entry/syscalls/syscall_32.tbl
 - Note that system call tables can differ between architectures
- You can run using the table values themselves using the `syscall()` wrapper
 - Q: why does `syscall()` exist?
 - If you're interested... There are debates
<https://lwn.net/Articles/771441/>

A driver crashes (5 min)

- A USB driver has a bug: it dereferences a null pointer.
- Work through what happens under **each** of the three designs:
 - Monolithic
 - Microkernel
- For each one:
 - What happens to the rest of the system?
 - What does the user see, and what do they have to do about it?
 - Who has to be trusted for this to go well?
- Follow-up: does it change anything if the monolithic kernel loaded the driver as a **module** instead of compiling it in?

What isn't clear?

Comments? Thoughts?